ChatGPT

clinicalclaw：面向医院的模块化临床AI执行与集成平台方案

执行摘要

clinicalclaw 旨在把“临床问题 → 可审计的工具链执行 → 可回写的结构化结果”做成一个可扩展的平台，而不是再做一个“能聊天的临床助手”。其核心灵感来自 Stanford University [1] 团队开源的 Biomni（你称为 Stanford BioMNI）：以“统一行动空间（action space）+ 规划检索 + 代码/工具执行”的方式，让一个通用 agent 在大量工具/数据库/软件上做组合式工作。Biomni 论文与官网明确了这一思想：Biomni-E1 通过挖掘大量文献构建统一行动空间，并集成 150 个专用工具、105 个软件包、59 个数据库；Biomni-A1 通过检索选择相关工具并生成逐步计划，再用可执行代码完成复杂工作流。[2]

但 Biomni 的安全模型与临床生产环境存在天然冲突：其 GitHub 文档警告“当前会以完整系统权限执行 LLM 生成代码”，并强调在生产需隔离/沙箱；同时注明公开工具不用于 PHI。[3] 因此 clinicalclaw 在继承“统一行动空间 + 规划执行”的同时，把**安全/合规/集成**上升为第一原则：默认最小权限、强审计、可回滚的人在回路（human-in-the-loop）、可在院内部署并对接 EPIC 与 PACS，以真实临床数据闭环。

在“claw 系”框架选择上，建议以 **x1jiang/clawagents_py（ClawAgents）作为内核**，原因是：它提供可插拔沙箱、工具白名单/黑名单、技能（skills）渐进加载、HTTP/SSE 网关、lane-based 并发队列与子代理机制，且是 Python 生态，便于快速对接 FHIR/DICOM、影像处理工具链与医院内部基础设施。[4] OpenClaw/NanoClaw/ZeroClaw 将作为参考实现：OpenClaw 强在“网关控制平面 + 多 agent 路由 + 安全默认策略 + 沙箱模式”；NanoClaw 强在“每个 agent 微 VM / 容器级隔离”；ZeroClaw 强在“可交换/可移植运行时”。[5]

30 天（2026-03-18 至 2026-04-16）目标是交付一个可演示的 MVP：
1) 以 SMART on FHIR 方式从 EPIC 启动（或通过后端服务模式连接）并拉取患者上下文；[6]
2) 通过 DICOMweb 从 PACS（或代理）检索/拉取影像（QIDO/WADO），并能以 STOW-RS 回传派生对象或报告；[7]
3) 将结果以 FHIR DiagnosticReport/Observation/DocumentReference 形式回写或导出，并保留 Provenance/AuditEvent 级别审计线索；[8]
4) 全链路日志、监控、权限、配置与可移植部署（on-prem / cloud / hybrid）。

临床痛点与产品定位

临床一线的“痛点”并不缺模型，而是缺一个**把院内数据与工具安全打通**的执行系统。clinicalclaw 的定位是“临床 AI 执行与集成层（Clinical AI Execution & Integration Layer）”，面向医院/医生解决以下典型问题：

信息碎片化与检索成本：诊前需要跨系统查既往病史、检查、报告、用药、关键趋势；影像在 PACS，报告在 EHR，量化指标散落在 PDF/文本/不同模块。FHIR 的 ImagingStudy 作为“影像元数据桥梁”可连接 Patient/

Encounter/ServiceRequest，并与影像系统通过 DICOM UID 对齐；DocumentReference 可作为临床文档/二进制对象索引。 [9]

影像 AI 的“最后一公里”：研究工具/模型很多，但临床落地需要稳定的数据接入（DICOM）、审计、任务状态、结果回写，而不是一次性脚本。DICOMweb 明确了以 REST 方式对 DICOM Study/Series/Instance 的 Query/Retrieve/Store（QIDO/WADO/STOW）能力，适合作为临床系统集成层的标准接口。 [7]

合规压力与安全风险：临床数据属于 ePHI，系统必须遵守 HIPAA Security Rule 的管理/物理/技术保障要求，并落实“minimum necessary”原则与去标识化路径（safe harbor / expert determination）。 [10] 现实中，agent/插件生态也存在供应链风险：例如 OpenClaw 的技能市场曾被报道出现大量恶意扩展，提示“插件化”必须内建签名、隔离与审计。 [11]

因此，clinicalclaw 的关键承诺是：

- **可扩展**：以“场景（Scenario）+ 工具（Tool）+ 连接器（Connector）+ 审计（Audit）”为最小模块组合，新增场景 ≈ 组合/新增少量插件，而不是改核心。
- **可插拔**：工具统一注册、统一参数 schema、统一权限策略、统一执行沙箱；支持引入院内现有工具链（影像处理、质控、检索、排程）。
- **可集成**：对接 EPIC 的 SMART on FHIR/OAuth2 流程与 FHIR API；对接 PACS 的 DICOMweb（或以 DICOMweb 作为 DIMSE 代理的现代入口）。 [12]
- **可审计可验证**：每一步输入/输出、工具调用、模型版本、操作者、时间、数据来源都能追溯（Provenance/AuditEvent），并可做回归测试。 [13]

基线与框架选择

Biomni 对 clinicalclaw 的启发与“临床化改造点”

Biomni 的核心贡献是把“行动空间”系统化：environment（E1）中集成大量工具/数据库/软件，agent（A1）再通过检索与计划将其组合执行，从而避免为每个任务硬编码流程。 [14] 其 E1 规模（150 工具、105 软件、59 数据库）给 clinicalclaw 一个直接启发：临床同样需要一个“院内行动空间”，但行动空间的对象不是公共数据库，而是：

- EHR（FHIR）资源读取/写入、上下文同步（SMART/可选 FHIRcast）； [15]
- 影像（DICOM）检索/拉取/回写； [16]
- 院内工具链（质控、分割、报告模板、术语映射、注册/去标识等）。

但 Biomni 文档也给出风险边界：执行 LLM 生成代码时具备“full system privileges”的安全警告，以及“public tool not for PHI”的限制。 [3] clinicalclaw 必须在架构层面将其“临床化”：

- 默认以**配置化工具调用**替代“任意代码执行”；
- 必要的计算在容器/沙箱内完成，并限制网络、文件、凭据；
- 数据最小化、分区、强审计；
- 输出永远需要临床审阅确认（尤其涉及诊断/治疗建议）。

claw 框架对比与选择建议

下表聚焦你指定的四类基线：openclaw、x1jiang/clawagents_py、nano claw（以 NanoClaw 为代表）、zero claw（ZeroClaw）。所有特性均以其官方仓库描述为准。

维度	openclaw	x1jiang/clawagents_py（ClawAgents）	NanoClaw	ZeroClaw
主要定位	多渠道个人/团队 agent，网关控制平面，支持多 agent 路由与 skills 平台 [17]	精简但“全栈”的 Python agent 框架：内置工具、技能系统、网关、沙箱、并发 lane、子代理 [18]	“可理解+强隔离”的 OpenClaw 替代：每个 agent 在容器/微 VM 中运行，强调 OS 级隔离 [19]	Rust 运行时，“deploy anywhere, swap anything”，强调可移植与可交换组件 [20]
沙箱/隔离	提供沙箱模式（如非主会话可在 Docker 沙箱中运行）及 DM pairing 等默认安全策略 [17]	提供可插拔沙箱后端（Local / InMemory）与工具白名单/黑名单、hook 机制 [21]	“Agents run in containers... micro VM” 强调容器级隔离与按组隔离上下文/文件系统 [19]	强调 secure-by-default runtime 与 trait-driven 可插拔架构（细节需进一步按其 docs 落地） [20]
网关/API 能力	Gateway WS control plane、CLI、doctor、渠道适配等完善 [17]	FastAPI 网关（/chat、/chat/stream、/queue、/health）+ lane 并发模型 [22]	以消息渠道为主的运行形态（WhatsApp/Telegram 等），强调轻量+可定制 [19]	以运行时基础设施为核心（需二次开发构建临床网关/连接器） [20]
插件/skills 模型	skills 平台（bundled/managed/workspace skills），但生态存在供应链风险，需要额外治理 [23]	skills 渐进加载（use_skill），自定义工具协议与 LangChain tool adapter [24]	鼓励“核心极简，扩展用 skills”；并强调可审计的小代码库 [19]	“pluggable everything”，但临床插件治理机制需自建 [20]
临床集成适配成本（30 天）	TypeScript 生态/功能复杂度较高；临床只用其一部分会“重装系统” [17]	Python 生态最利于 FHIR/DICOM/影像工具链；内置网关与权限钩子降低 MVP 成本 [25]	隔离强但偏“消息驱动个人助手”范式；临床场景需要 UI/审计/回写模块更多二开 [19]	Rust 迁移成本高；适合作为长期高性能运行时而非 30 天 MVP [20]

结论（推荐）：以 ClawAgents 作为 clinicalclaw 的核心 agent 运行时（最小可行），并借鉴 OpenClaw/NanoClaw 的两类关键经验：
1) **控制平面思维**（会话/路由/队列/审计统一入口），OpenClaw 已将其写进 Gateway 与 session 模型； [17]
2) **OS 级隔离与最小权限**（NanoClaw 强调每 agent 容器/微 VM，避免“单进程共享一切”的风险）； [19]
再结合对插件供应链风险的现实教训，为 clinicalclaw 从第一天起设计**插件签名、隔离、审核与回滚**。 [26]

推荐总体架构

模块化组件设计

clinicalclaw 建议采用“平台内核 + 连接器层 + 场景层 + 工具层 + 审计与观测层”五层结构：

- **平台内核（Core Runtime）**：
  以 ClawAgents 的 ReAct loop、工具协议、skills、子代理与 FastAPI 网关为基础；并使用其 lane-based 并发队列区分主交互、定时任务、子代理、嵌套调用，避免并发引发状态污染。[18]

- **连接器层（Connectors）**：

- EHR：SMART on FHIR/OAuth2 启动与鉴权（EHR launch 含 iss/launch），并调用 FHIR API；EPIC 明确支持 OAuth2/SMART on FHIR。[27]
- 影像：对 PACS/VNA 主要走 DICOMweb（QIDO/WADO/STOW）；DICOMweb 被定义为 DICOM 的 web RESTful 服务集合，亦可作为 DIMSE 代理入口。[28]

- **场景层（Scenario Library）**：
  将临床 workflow 固化为“可审计的任务模板 + 权限策略 + 工具集合”。类似 Biomni 避免硬编码单一流程，但临床更强调：每个场景都定义明确的**输入范围、允许工具、输出格式、审阅路径**。[29]

- **工具层（Tooling）**：
  工具以 registry 形式注册，统一 JSON schema 参数验证与类型强制；ClawAgents 已支持工具协议与参数 schema。[30]

- **审计与观测层（Audit/Observability）**：

- 结构化审计：用 FHIR Provenance 描述“实体/过程如何生成资源”，用 AuditEvent 记录“who/what/where/when/why”事件。[31]
- 运行观测：日志、指标、追踪（可与院内 SIEM/监控体系对接）。

架构示意图（Mermaid）

```
flowchart LR
  subgraph Clinician
    UI[SMART-on-FHIR 应用 / 医生门户]
  end

  subgraph Hospital Systems
    EHR[EPIC / 其他 EHR 的 FHIR 端点]
    PACS[PACS/VNA]
    DICOMWEB[DICOMweb 端点或代理]
  end
```

```
  subgraph clinicalclaw
    GW[API Gateway (FastAPI)]
    RT[Claw Runtime + Scenario Router]
    TR[Tool Registry + Policy Engine]
    SB[Sandbox Executor (容器/VM/受限网络)]
    DB[(PostgreSQL 元数据)]
    OBJ[(对象存储/影像工件存储)]
    AUD[Audit & Provenance]
    OBS[Logs/Metrics/Tracing]
  end

  UI -->|SMART Launch + OAuth2| EHR
  UI -->|任务请求| GW --> RT
  RT --> TR --> SB
  SB -->|FHIR Read/Write| EHR
  SB -->|QIDO/WADO| DICOMWEB --> PACS
  SB -->|STOW (可选回写)| DICOMWEB

  SB --> DB
  SB --> OBJ
  RT --> AUD
  GW --> OBS
  RT --> OBS
  SB --> OBS
```

工具集成模式、Agent 类型与数据标准

工具集成模式：Tool Registry + Adapter + Sandboxing

clinicalclaw 的“可插拔”落地建议采用三件套：

1) **Tool Registry（工具注册表）**
每个工具具备：name、description、parameters（JSON schema）、execute()。ClawAgents 的自定义工具协议与 schema 示例可直接复用。[30]
关键扩展：加入 `data_classification`（PHI/非 PHI）、`requires_break_glass`、`network_policy`、`write_back` 等元数据，用于策略引擎决策。

2) **Adapter（适配器）**
- EHRAdapter：封装 SMART on FHIR/OAuth2、token 生命周期、FHIR read/search/create/update；EPIC on FHIR 文档明确描述 EHR Launch、iss/launch 参数与 OAuth2 过程，可作为实现依据。[32]
- DICOMAdapter：封装 QIDO-RS（查 Studies/Series/Instances）、WADO-RS（取像/元数据/像素）、STOW-RS（存实例）。[33]

3) **Sandboxing（沙箱执行）**
- ClawAgents 自带“可插拔沙箱后端”用于文件/命令操作，并支持 allow_only_tools / block_tools 与 before_tool hooks，适合在临床环境做最小权限化。[21]
- 但临床计算（影像推理/质控）建议进一步升级为 **容器级沙箱**：借鉴 NanoClaw 的思路，“agents run in containers... only access explicitly mounted directories”。[19]
- 供应链治理：鉴于 OpenClaw 技能市场出现过大规模恶意 add-ons 的报道，clinicalclaw 的插件必须支持签名、审核、隔离与回滚（不是“装了就能跑”）。[26]

Agent 类型设计：vibe/codex + 临床工具型 agent

clinicalclaw 中建议明确区分“研发加速 agent”和“临床执行 agent”，避免把“会写代码的 agent”直接放进生产碰 PHI。

- **vibe agent（需求与场景 agent）**：
  用于与临床专家/信息科访谈，把痛点转成“场景规范（Scenario Spec）”：输入、输出、允许操作、审阅点、失败降级策略。产出的是文档/配置，不是直接执行。

- **codex agent（开发与集成 agent）**：
  用于生成 plugin/connector 的代码骨架、单元测试、文档与 CI 配置；但必须在无 PHI 的开发环境运行，并通过代码审查与安全扫描后再发布到插件仓库（内部制品库）。

- **临床工具型 agents（生产）**：
  使用 ClawAgents 的 subagent 机制定义“专职 agent”，并通过工具白名单做到职责隔离：ClawAgents 已支持注册具名 sub-agents。[34]
  推荐最小集合：
  1) Orchestrator（总控/路由）：只做意图识别、场景选择、任务拆分，不直接读写外部系统。
  2) EHR Agent：只允许 FHIR read/search + 受控写入（如写回 DocumentReference）。
  3) Imaging Agent：只允许 DICOMweb QIDO/WADO + 影像计算沙箱。
  4) Safety/Audit Agent：只读审计与轨迹，生成合规报告与异常告警（不触碰 PHI 内容本体，只处理元数据/哈希/事件）。

数据标准与预处理：FHIR + DICOM（可选 BIDS）

FHIR 侧（临床数据/工作流/可追溯）
- ImagingStudy：用于表示 DICOM Study 的元数据，并与 Patient/ServiceRequest/Encounter 等资源建立关系。[35]
- DiagnosticReport：实验室/病理/影像报告等“诊断报告”推荐建模资源，并可引用 Observation。[36]
- Task：用于跟踪工作流任务状态与执行进度；FHIR 工作流明确以 Task 表示 workflow state。[37]
- DocumentReference：用于索引文档/临床笔记/二进制对象（含图片、PDF 等）。[38]
- Provenance：用于记录资源生成/传递/影响的实体与过程，是可信与可复现基础。[39]
- AuditEvent：用于记录安全/隐私/操作相关事件（who/what/where/when/why）。[40]
- Consent（可选）：FHIR Consent 支持 permit/deny 与例外条款结构，可作为“数据共享/研究使用”治理的一部分（但各院落地差异大，MVP 可先不强依赖）。[41]

DICOM 侧（影像数据）
- DICOMweb：DICOM 的 web RESTful 标准，可直接实现或作为 DIMSE 的代理，实现现代访问。[28]

- QIDO-RS：查询 Studies/Series/Instances； [42]
- STOW-RS：存储实例（便于回写派生对象）； [43]

（可选）研究/批处理预处理：BIDS
若你的首个落地场景偏神经影像队列/随访研究，可在沙箱中将 DICOM 转为内部 BIDS 结构以获得可复现处理；BIDS 明确提供一致的数据组织方式。 [44]
注意：一些工具默认会提交指标（如 MRIQC 提到 IQMs 默认提交，可用 `--no-sub` 禁用），临床/院内环境应默认关闭任何外发。 [45]

隐私、合规、安全、验证与部署

HIPAA 视角的安全与数据治理要点

clinicalclaw 的“可上院内”的关键不在模型，而在治理与控制：

- **HIPAA Security Rule** 要求对 ePHI 实施合理且适当的管理/物理/技术保障。 [46]
- **Minimum Necessary**：当适用时，应将 PHI 的使用/披露/请求限制在达成目的所需的最小范围。 [47]
- **去标识化**：HIPAA 提供 safe harbor 与 expert determination 两条路径；组织应依据 45 CFR 164.514 建立可审计流程。 [48]

对应到架构的强制落地项：

- **最小权限工具集**：每个场景声明 allowed-tools；运行时强制 whitelist（ClawAgents allow_only_tools）并支持 block_tools 与 before_tool 拦截。 [49]
- **凭据隔离与密钥管理**：FHIR token、PACS 访问密钥只注入到需要的沙箱，严禁进入 LLM 上下文与日志正文（只记录哈希/引用）。
- **出网控制（egress control）**：生产沙箱默认无公网；如需知识检索，使用院内代理/白名单域名并对返回内容做净化。
- **审计默认开启**：对每次读写 EHR/PACS、每个工具调用、每份输出生成 AuditEvent；对每份写回资源附 Provenance 记录（至少包括 agent、时间、输入来源、算法/模型版本）。 [50]
- **插件治理**：插件必须为“内部制品库 + 签名 + 安全扫描 + 人审”。OpenClaw 技能市场的恶意扩展事件说明“开放市场 + 高权限工具”组合是高风险。 [26]

验证与测试策略

目标：把“临床可用”拆成可验证的工程指标，而非主观体验。

- **接口一致性测试**
- FHIR：对关键资源（ImagingStudy/DiagnosticReport/Observation/DocumentReference/Task）做 schema 验证与回归测试。 [51]

- DICOMweb：对 QIDO/WADO/STOW 的最小能力做集成测试（可用院内 DICOMweb 或代理）。 [52]

- **安全测试（必做）**

- 工具白名单绕过测试（prompt injection / tool misuse）
- 插件供应链测试（签名校验、SBOM、依赖漏洞扫描）
- 日志脱敏测试（确保 token/PHI 不落盘）
- **临床人在回路测试（MVP 即引入）**
  每个场景输出都进入“审阅队列”，由医生确认后才允许回写 EHR（或生成正式文档）。FHIR Task 资源适合表达任务执行与状态追踪。[37]

部署方式：on-prem / cloud / hybrid

- **On-prem（优先建议）**：数据不出院/不出专网，最容易通过合规审查；模型可选院内部署或通过医院允许的外部服务（如已签 BAA/合规边界）。HIPAA 的技术保障与最小必要原则在 on-prem 更易工程化执行。[53]
- **Hybrid**：EHR/PACS 在院内，非 PHI 的模型评估/插件构建在云端；或影像计算在院内 GPU，元数据/任务编排可在云。
- **Cloud（以标准接口为基础）**：若医院允许将影像数据放入云端 DICOM 服务，许多云厂商以 DICOMweb（QIDO/WADO/STOW）提供托管能力：
- Microsoft[54] 的 Azure DICOM service 支持 DICOMweb 标准 API（STOW/QIDO/WADO）等能力；[55]
- Google[56] Cloud Healthcare API 的 DICOM store 支持 DICOM PS3.18 DICOMweb（WADO-RS/STOW-RS/QIDO-RS）。[57]
- Amazon Web Services[58] 的 HealthImaging 提供对 WADO-RS 风格的检索/获取能力。[59]

  MVP 阶段建议：默认按 on-prem/hybrid 设计，云端选项仅作为“可迁移接口层”保留（核心在统一 DICOMweb/FHIR 语义与 tool registry）。

交付物、MVP 清单与 30 天落地计划

交付物列表（面向合作者/医院）

到 2026-04-16（30 天）建议对外交付以下资产：

1) **clinicalclaw MVP 系统**（可部署、可演示、可审计）
- SMART on FHIR 启动（EPIC sandbox 可跑通）+ 任务 UI
- DICOMweb 影像拉取（以院内 DICOMweb 或代理环境跑通）
- 场景引擎（至少 2 个场景）+ 审阅队列
- 审计日志（AuditEvent）+ 结果血缘（Provenance）

2) **插件开发套件（SDK）与示例库**
- Tool/Adapter 模板
- 插件打包、签名、发布流程
- 最小安全基线（扫描/测试）

3) **合作指南与合规包（初版）**
- 数据流与最小必要说明（面向信息科/合规/IRB）
- 风险评估与缓解清单
- 试点落地 SOP（账号/权限/网络/审计/回滚）

MVP 优先级功能清单

P0（必须做，决定能否试点）
- 统一网关与会话：基于 ClawAgents 网关（/chat、/chat/stream、/queue、/health）封装 clinicalclaw API。[22]
- 工具注册表 + 策略引擎：每场景 allowed-tools、PHI 标识、写回开关。
- EPIC 集成：SMART on FHIR/OAuth2 EHR Launch 跑通（iss/launch）。[32]
- PACS 集成：DICOMweb QIDO/WADO 拉取影像元数据/实例；STOW 先做"可选回写/模拟回写"。[33]
- 输出：至少支持生成 DiagnosticReport + DocumentReference（HTML/PDF/JSON 工件），并附 Provenance。[60]
- 审阅队列（human-in-the-loop）与任务状态（Task）。[37]
- 审计：所有外部系统访问生成 AuditEvent。[40]

P1（强烈建议，显著提升可信度）
- 容器级沙箱执行（参考 NanoClaw 思路）：影像计算与任何"可执行工具"在容器/微 VM 内跑。[19]
- 插件签名/审核/回滚流水线（吸取 OpenClaw skills 供应链风险教训）。[26]
- 结果回写：将派生对象以 STOW-RS 回写到 DICOMweb（或回写到 EHR 文档）。[43]

P2（30 天内尽量不扩张，除非资源充足）
- FHIRcast 上下文同步（与影像 viewer/多应用协同），FHIRcast 建议用 SMART 做 session discovery，并支持 WebSocket 通知。[61]
- OHIF viewer 深度集成（若需要浏览器影像查看）：OHIF 对 DICOMweb 有开箱支持。[62]

关键组件清单表（工程拆解）

组件	目的	最小实现（30 天）	通过标准/依据
Agent Runtime	工具调用、子代理、skills	采用 ClawAgents（工具协议+skills+网关+沙箱）[63]	ClawAgents 设计：工具/skills/沙箱/网关 [64]
EHR Connector	拉取临床上下文、回写结果	SMART on FHIR/OAuth2 + FHIR read/search + 受控写入 [32]	HL7 SMART App Launch（OAuth2 模式）[65]
PACS Connector	影像检索/拉取/回写	DICOMweb QIDO/WADO；STOW 作为可选回写	DICOMweb/QIDO/STOW 标准 [7]
Workflow/Queue	状态、并发、回滚	lane-based 队列 + Task 状态机	FHIR Task 用于追踪执行状态 [37]
Audit/Lineage	合规与可追溯	AuditEvent + Provenance 记录每次访问与输出	FHIR AuditEvent/Provenance 定义 [50]

组件	目的	最小实现（30 天）	通过标准/依据
Artifact Store	存放报告/派生文件	对象存储（S3/MinIO）+ metadata DB	DocumentReference 作为索引 [38]
Sandbox Exec	限权执行	容器/VM + 只挂载必要目录 + 无公网	NanoClaw 类型隔离理念 [19]
Observability	运维与质量	结构化日志 + 指标 + trace（MVP 可先日志+指标）	NIST 控制目录用于安全/隐私控制参考 [66]

30 天 Gantt 风格计划（2026-03-18 → 2026-04-16）

```
gantt
  title clinicalclaw 30天计划（2026-03-18 至 2026-04-16）
  dateFormat  YYYY-MM-DD
  axisFormat  %m-%d

  section 基座与研究
标准与基线研读（SMART/DICOMweb/HIPAA/ClawAgents）  :a1, 2026-03-18, 5d
架构与威胁模型（威胁面/最小权限/审计字段）         :a2, 2026-03-20, 6d

  section 核心平台
项目脚手架（FastAPI网关+ClawAgents集成+DB模型）    :b1, 2026-03-18, 7d
  Tool Registry + Policy Engine（schema/allowlist）  :b2, 2026-03-22, 8d
审计与轨迹（AuditEvent/Provenance/日志规范）       :b3, 2026-03-24, 8d

  section 系统集成
  EPIC侧：SMART on FHIR/OAuth2 + FHIR读取            :c1, 2026-03-25, 8d
  PACS侧：DICOMweb(QIDO/WADO) 拉取 + 工件存储        :c2, 2026-03-28, 9d
回写：DocumentReference/DiagnosticReport（最小）     :c3, 2026-04-02, 6d

  section 场景与插件
场景规范（2-3个）+ 技能库（skills）                  :d1, 2026-03-22, 10d
影像计算插件（先stub，后可接MRIQC/FastSurfer）        :d2, 2026-04-01, 8d
容器沙箱执行与插件签名（P1）                         :d3, 2026-04-05, 8d

  section 测试与试点
集成测试（Epic sandbox + DICOMweb测试端点）           :e1, 2026-04-06, 6d
安全测试（prompt/tool misuse/secret leak）            :e2, 2026-04-08, 5d
试点演示包与文档（SOP/部署/回滚/指标）                :e3, 2026-04-11, 6d
```

每周里程碑与每日必做清单（高强度 >8h/day）

Week 1（03/18–03/24）目标：可运行的“空壳平台” + 2 个场景草案
- 你必须完成的编码任务：

- 建 repo + CI（lint/test）+ 基础 FastAPI 服务骨架
- 集成 ClawAgents：网关调用、工具注册、allow_only_tools/ block_tools 机制暴露为平台策略 API [67]
- 定义核心数据模型（Case/TaskRun/Artifact/AccessEvent），并设计“审计字段最小集”映射到 AuditEvent/Provenance [50]
- 工具发现任务：整理院内常见“工具需求清单”（见下文 Outreach），并把每类工具抽象成 ToolSpec（输入/输出/权限）

Week 2（03/25–03/31）目标：EPIC sandbox 拉通 + DICOMweb 拉通（读）
- 你必须完成的编码任务：
- SMART on FHIR EHR Launch：处理 iss/launch，完成 token 交换与 FHIR 读取（至少 Patient/Encounter/ImagingStudy 或 DiagnosticReport 查询路径） [68]
- DICOMweb：实现 QIDO-RS 搜索 + WADO-RS 拉取实例/元数据；把影像实例落到对象存储并写 metadata DB [69]
- 场景定义任务：确定 MVP “首发场景” 1 个（见建议），并写成 Scenario Spec v1（允许工具、输出、审阅）

Week 3（04/01–04/07）目标：输出可交付（报告 + 审阅队列）
- 你必须完成的编码任务：
- 输出生成：DiagnosticReport + DocumentReference（先以 HTML/PDF/JSON 工件形式） [70]
- 引入 Task 状态机与审阅队列（pending→in_review→approved→filed / rejected） [37]
- 审计：所有外部访问自动写 AuditEvent，输出写 Provenance（至少包含 agent、时间、输入引用） [50]

Week 4（04/08–04/16）目标：端到端演示 + 安全基线 + 试点资料
- 你必须完成的编码任务：
- 容器级沙箱执行（至少影像计算插件；若算法未就绪，用 stub 但保留接口与资源限制） [19]
- 插件治理：签名校验、版本锁定、回滚（哪怕先做“内部插件仓库 + 校验和”最小版）
- 集成测试与压测：并发 lane 与队列安全性（ClawAgents lanes） [71]
- 你必须完成的“对外任务”：完成 2 家潜在数据/临床合作方的沟通推进（至少拿到 1 个“去标识 DICOM + 对应 FHIR 元数据”的试点数据包）

首发场景建议（30 天可落地）

为满足“EPIC + PACS + 真实数据”闭环，同时避免触碰监管灰区，建议 MVP 选择偏“流程提效 + 量化辅助 + 医生审阅”的场景，而非“自动诊断”。

场景 A：影像+病历“诊前准备包”
- 输入：医生在 EPIC 中打开患者，启动 clinicalclaw；系统通过 FHIR 拉取核心信息（近期主诉/问题列表/用药/关键检验/既往影像检查清单）。 [72]
- 影像：通过 DICOMweb 查询近 6–12 个月同部位影像 Study，并拉取关键序列或缩略图/元数据。 [69]
- 输出：生成一份可审阅的“结构化诊前摘要”（DocumentReference + 可选 DiagnosticReport），并附血缘/审计。 [73]

场景 B：影像质控与可复现计算（研究/随访队列）
- 输入：选定一组检查（ImagingStudy 列表），批量拉取影像并跑 QC/派生指标。 [74]
- 工具：可接 MRIQC（注意关闭默认指标提交）与 FastSurfer 等容器化工具；FastSurfer 将分割与表面重建拆成子流水线，适合作为可插拔计算模块。 [75]
- 输出：以 Observation（数值）+ DocumentReference（报告）形式交付，并进入审阅。 [70]

30 天内不要做：自动下诊断/自动改医嘱/自动发患者沟通信息。先把“数据接入—可审计执行—可回写报告—医生审阅”跑通。

风险、缓解策略与成功指标

主要风险与缓解（务实版）

1) **拿不到真实院内数据/接口权限**（最常见）
- 缓解：EPIC sandbox 先把 SMART/FHIR 流程跑通；影像侧用 DICOMweb 测试端点或院内代理环境先完成接口；并行推进真实合作（见 outreach 模板）。 [76]

2) **PACS 不提供 DICOMweb 或策略限制**
- 缓解：DICOMweb 标准本身支持“作为 DIMSE 代理”实现；可与影像科/厂商协商开通 DICOMweb 网关或使用代理方案。 [77]

3) **插件/技能供应链风险导致安全事故**
- 缓解：内部插件仓库 + 强签名 + 扫描 + 人审 + 运行时隔离；不要直接消费公共技能市场。参考 OpenClaw 扩展被报道的恶意技能事件作为反例。 [26]

4) **LLM/agent 行为不可控**（尤其写回）
- 缓解：写回必须 gated（审批 + 双人确认可选）；工具白名单；对 write-back 工具增加“结构化输出 schema + 规则校验 + 影子模式（shadow mode）”。ClawAgents 的 allow_only_tools 与 hooks 机制可作为拦截点。 [49]

5) **合规审查周期远超 30 天**
- 缓解：把 30 天目标定义为“技术可行性 MVP + 合规包初版”，并用去标识样本数据做演示；真实 ePHI 试点作为下一阶段里程碑。

成功指标（可量化、可验收）

集成指标
- SMART on FHIR：EPIC sandbox 中 EHR launch 成功率 ≥ 95%，token 刷新/过期处理正确。 [32]
- DICOMweb：QIDO 查询 + WADO 拉取成功率 ≥ 95%；对 STOW 的最小回写接口可用（即使先写到测试库）。 [33]

安全与合规指标
- 0 明文密钥落盘；日志通过自动测试验证不含 token。
- 每次外部访问均产生 AuditEvent；每份输出均有 Provenance。 [50]
- 场景级“最小必要”：每个场景有数据字段白名单与工具白名单，并可导出审计报告。 [47]

临床价值指标（试点期）
- 场景 A：医生诊前准备时间减少（自报）≥ 20%（初期可用问卷/N=5–10）。
- 场景 B：同一队列重复分析的可复现率（同输入→同输出哈希）≥ 99%。

对外合作与数据获取模板（医院/数据伙伴）

模板一：联系医院临床牵头人（科主任/专科医生/临床研究 PI）

主题：院内试点合作提案｜clinicalclaw：可审计、可插拔的临床AI执行平台（对接 EPIC + PACS）

您好，老师：
我在构建一个面向医院内部署的临床 AI 平台 clinicalclaw，目标不是“聊天”，而是把临床常见任务做成**可审计、可回写、可扩展**的执行工作流：从 EPIC（SMART on FHIR）获取患者上下文，从 PACS（DICOMweb）拉取影像，再通过插件化工具链生成结构化报告并进入医生审阅后回写。平台会将所有读写行为记录为审计事件（AuditEvent）并附带结果血缘（Provenance），以满足院内合规与可追溯需求。 [78]

希望与贵科共同定义 1–2 个高价值场景做 4–8 周试点（我当前 30 天完成 MVP）。我方期望贵科支持：
- 指定 1 名临床 champion（每周 30–60 分钟沟通需求与评审输出）
- 提供去标识样本数据或在院内受控环境中进行影子模式验证（不直接写回）
- 协助对接信息科/影像科以确认接口可用性与审批流程

如您方便，我希望本周约 30 分钟线上沟通：
1) 哪些流程最耗时、最碎片化；2) 输出格式与审阅点；3) 试点数据与合规路径。
谢谢！

模板二：联系信息科/集成团队（FHIR/接口/应用商店负责人）

主题：请求技术对接｜clinicalclaw SMART on FHIR（EPIC）+ DICOMweb（PACS）试点最小权限方案

您好：
我们计划在院内开展 clinicalclaw 试点。该系统以 SMART on FHIR/OAuth2 从 EPIC 启动并读取 FHIR 资源（EHR Launch 流程、iss/launch 参数），并通过 DICOMweb（QIDO/WADO/STOW）对接影像系统；全程最小权限与审计（AuditEvent/Provenance）。 [79]

我们希望确认：
- 是否可注册一个 provider-facing SMART app（测试环境优先），以及所需 scopes 与审查流程；SMART App Launch IG 以 OAuth2 为基础。 [65]
- 是否存在院内 DICOMweb 端点或推荐代理方式（如 PACS 厂商模块）；DICOMweb 可直接实现或作为 DIMSE 代理。 [77]
- 审计/日志对接要求（是否需映射到现有 SIEM）

我们可先在 Epic sandbox 完成验证，再迁移至院内测试环境。请问贵方最合适的技术联系人与申请流程是？

模板三：联系影像科/PACS 管理员（DICOMweb/数据回写）

主题：PACS 对接需求（DICOMweb）｜QIDO/WADO 拉取 + STOW 回写派生对象（测试环境）

您好：
我们在院内试点 clinicalclaw，需要通过 DICOMweb 标准接口对 PACS/VNA 做：

- QIDO-RS：查询 Studies/Series/Instances； [42]
- WADO-RS：拉取影像实例与元数据； [80]
- STOW-RS（可选）：回写派生对象/报告（先在测试库验证）。 [43]

请问贵方：
1) 是否已有 DICOMweb 端点或厂商网关模块；2) 测试环境开通流程；3) 访问控制与审计要求；4) 是否允许回写（若不允许，我们也可只读）。
我们会在受控网络/容器沙箱中运行计算，并仅挂载必要数据路径，确保最小权限。

附注（面向合作者的现实约束声明）：clinicalclaw 在 30 天内的目标是把“标准接口 + 可插拔工具 + 安全审计 + 医生审阅”打通并可演示；真实临床 ePHI 的深度试点需要走医院既有的数据治理与权限流程（minimum necessary、去标识化、访问审计），并将在 MVP 后以合规包与影子模式逐步推进。 [81]

---

[1] [40] [50] https://build.fhir.org/auditevent.html
https://build.fhir.org/auditevent.html

[2] [14] [29] https://pmc.ncbi.nlm.nih.gov/articles/PMC12157518/
https://pmc.ncbi.nlm.nih.gov/articles/PMC12157518/

[3] https://github.com/snap-stanford/Biomni
https://github.com/snap-stanford/Biomni

[4] [18] [21] [22] [24] [25] [30] [34] [49] [63] [64] [67] [71] https://github.com/x1jiang/clawagents_py
https://github.com/x1jiang/clawagents_py

[5] [17] [23] https://github.com/openclaw/openclaw
https://github.com/openclaw/openclaw

[6] [12] [27] [32] [68] [72] [78] [79] https://fhir.epic.com/Documentation?docId=oauth2
https://fhir.epic.com/Documentation?docId=oauth2

[7] [16] [28] [56] [77] [80] https://www.dicomstandard.org/using/dicomweb
https://www.dicomstandard.org/using/dicomweb

[8] [36] [60] [70] https://build.fhir.org/diagnosticreport.html
https://build.fhir.org/diagnosticreport.html

[9] [35] [51] [74] https://build.fhir.org/imagingstudy.html
https://build.fhir.org/imagingstudy.html

[10] [46] [53] https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html
https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html

[11] [26] [54] https://www.theverge.com/news/874011/openclaw-ai-skill-clawhub-extensions-security-nightmare
https://www.theverge.com/news/874011/openclaw-ai-skill-clawhub-extensions-security-nightmare

[13] [31] [39] https://build.fhir.org/provenance.html
https://build.fhir.org/provenance.html

15 65 https://build.fhir.org/ig/HL7/smart-app-launch/
https://build.fhir.org/ig/HL7/smart-app-launch/

19 58 https://github.com/qwibitai/nanoclaw
https://github.com/qwibitai/nanoclaw

20 https://github.com/zeroclaw-labs/zeroclaw
https://github.com/zeroclaw-labs/zeroclaw

33 42 52 69 https://dicom.nema.org/medical/dicom/2018d/output/html/part18.html
https://dicom.nema.org/medical/dicom/2018d/output/html/part18.html

37 https://build.fhir.org/task.html
https://build.fhir.org/task.html

38 73 https://build.fhir.org/documentreference.html
https://build.fhir.org/documentreference.html

41 https://build.fhir.org/consent.html
https://build.fhir.org/consent.html

43 https://dicom.nema.org/medical/dicom/2016d/output/chtml/part18/sect_6.6.html
https://dicom.nema.org/medical/dicom/2016d/output/chtml/part18/sect_6.6.html

44 https://bids.neuroimaging.io/index.html
https://bids.neuroimaging.io/index.html

45 https://mriqc.readthedocs.io/en/latest/usage.html
https://mriqc.readthedocs.io/en/latest/usage.html

47 81 https://www.hhs.gov/hipaa/for-professionals/privacy/guidance/minimum-necessary-requirement/index.html
https://www.hhs.gov/hipaa/for-professionals/privacy/guidance/minimum-necessary-requirement/index.html

48 https://www.hhs.gov/hipaa/for-professionals/special-topics/de-identification/index.html
https://www.hhs.gov/hipaa/for-professionals/special-topics/de-identification/index.html

55 https://learn.microsoft.com/en-us/azure/healthcare-apis/health-data-services-get-started
https://learn.microsoft.com/en-us/azure/healthcare-apis/health-data-services-get-started

57 https://docs.cloud.google.com/healthcare-api/docs/dicom
https://docs.cloud.google.com/healthcare-api/docs/dicom

59 https://docs.aws.amazon.com/healthimaging/latest/devguide/dicomweb-retrieve.html
https://docs.aws.amazon.com/healthimaging/latest/devguide/dicomweb-retrieve.html

61 https://build.fhir.org/ig/HL7/fhircast-docs/2_Specification.html
https://build.fhir.org/ig/HL7/fhircast-docs/2_Specification.html

62 https://github.com/OHIF/Viewers
https://github.com/OHIF/Viewers

66 https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final
https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final

75 https://github.com/nipreps/mriqc
https://github.com/nipreps/mriqc

76 https://fhir.epic.com/

https://fhir.epic.com/